SIMATS ENGINEERING
CO3-ASSESSMENT TOOL 1
Experiments

COURSE NAME: Computer Networks **COURSE CODE: CSA07**

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

Code of Conduct:

I **G. GOWTHAMI (192524285)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: **G. GOWTHAMI**

Criteria	Understanding of Concepts 2.5	Experimental Design 3	Use of Tools 2	Analysis of Results 1.5	Documentation 1	Total (10)
Q1						
Q2						
Q3						
Q4						

Faculty Signature

Experiments

QUESTIONS:4 **Total Marks:40**

1. Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, and ACK). Demonstrate the role of each flag and how they establish a reliable connection. (BL4) (10)
2. Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability. (BL4) (10)
3. Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP. (BL4) (10)
4. Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP.(BL4) (10)

ANSWERS

1.TCP Three-Way Handshake – SYN, SYN-ACK and ACK

Aim

To capture packets during TCP connection setup using Wireshark and identify the SYN, SYN-ACK, and ACK packets involved in establishing a reliable connection.

Understanding of Concepts

TCP uses a three-way handshake to establish a reliable connection between a client and server.

The three packets are:

Step	Packet	Important Flag	Purpose
1	Client → Server	**SYN = 1**	Requests a TCP connection
2	Server → Client	**SYN = 1, ACK = 1**	Accepts request and synchronizes sequence numbers
3	Client → Server	**ACK = 1**	Confirms the server's response

Experimental Procedure

1. Open **Wireshark**.
2. Select the active network interface.
3. Start packet capture.
4. Open a website or establish a TCP connection.
5. Stop the capture.
6. Apply the filter:

tcp.flags.syn == 1

7. Identify the SYN and SYN-ACK packets.
8. Check the corresponding ACK packet.

Packet Analysis

Packet 1 – SYN

The client sends a SYN packet to the server.

Client → Server

SYN = 1

ACK = 0

It contains an initial sequence number and informs the server that the client wants to establish a connection.

Packet 2 – SYN-ACK

The server responds:

Server → Client

SYN = 1

ACK = 1

The server acknowledges the client's SYN and sends its own sequence number.

Packet 3 – ACK

The client sends:

Client → Server

SYN = 0

ACK = 1

This confirms that the client received the server's SYN-ACK. The TCP connection is now established.

Role of Flags

- **SYN:** Synchronizes sequence numbers and initiates a connection.
- **ACK:** Acknowledges successful receipt of data/control information.
- **FIN:** Used later to terminate a TCP connection.
- **RST:** Immediately resets/terminates a connection when necessary.

Analysis of Results

The three-way handshake ensures that **both sides are ready to communicate** and that their initial sequence numbers are synchronized. This provides the foundation for TCP's reliable, ordered communication.

Result

The TCP three-way handshake was successfully captured and the SYN, SYN-ACK, and ACK packets were identified. The role of each flag in establishing a reliable connection was verified.

2.DNS Query Using UDP – UDP vs TCP

Aim

To capture a DNS query using UDP in Wireshark and compare the UDP header with the TCP header.

Understanding of Concepts

DNS commonly uses UDP port 53 for normal queries because UDP has a smaller header and does not require connection establishment.

Experimental Procedure

1. Open Wireshark.
2. Start packet capture.
3. Open Command Prompt.
4. Execute:

nslookup google.com

5. Stop the capture.
6. Apply the filter:

dns

or:

udp.port == 53

7. Select the DNS query packet.
8. Expand the User Datagram Protocol section.

UDP and TCP Comparison

Feature	**UDP**	**TCP**
Header size	**8 bytes**	**20 bytes minimum**
Connection	Connectionless	Connection-oriented
Sequence number	No	Yes
Acknowledgment	No	Yes
Retransmission	No	Yes
Speed	Faster	Relatively slower
Reliability	Lower	Higher
DNS use	Commonly used	Used when required

UDP Header

The UDP header contains:

Source Port

Destination Port

Length

Checksum

It does **not** contain sequence numbers or acknowledgment numbers.

Analysis of Results

UDP's lightweight design reduces overhead and allows DNS queries to be sent quickly. However, because UDP does not provide acknowledgments or automatic retransmission, a lost DNS packet must be handled by the DNS application or resolver through mechanisms such as retrying the query.

TCP provides greater reliability but requires additional control information and connection management.

Result

The DNS query was successfully captured using UDP. The **8-byte UDP header** was observed, and the absence of sequence numbers and acknowledgments was verified.

3. TCP Retransmission vs UDP Packet Loss

Aim

To simulate packet loss and observe how TCP retransmits lost packets while UDP does not provide TCP-style retransmission.

Understanding of Concepts

TCP provides reliable delivery using:

- Sequence numbers
- Acknowledgments
- Retransmissions
- Flow control
- Congestion control

UDP does not provide these TCP reliability mechanisms.

Experimental Procedure

1. Open Wireshark.
2. Start capturing packets.
3. Establish a TCP connection.
4. Briefly disconnect or interrupt the network connection.
5. Restore the connection.
6. Stop packet capture.
7. Apply:

tcp.analysis.retransmission

8. Examine retransmitted TCP packets.

For UDP traffic, use:

udp

TCP Packet Loss

When a TCP packet is lost:

Sender → Data Packet → X

Sender ← ACK / Duplicate ACK

Sender → Retransmitted Packet

TCP detects loss using acknowledgments, duplicate ACKs, or retransmission timers and sends the missing segment again.

UDP Packet Loss

UDP does not automatically retransmit a lost packet:

Sender → UDP Packet → X

There is no TCP-style acknowledgment or automatic retransmission at the UDP transport layer.

Comparison

Feature	TCP	UDP
Packet acknowledgment	Yes	No
Sequence numbers	Yes	No
Automatic retransmission	Yes	No
Reliability	High	Lower
Overhead	Higher	Lower
Speed	Generally slower	Generally faster
Suitable for	Web, file transfer, email	DNS, streaming, real-time applications

Analysis of Results

The Wireshark capture demonstrates that TCP attempts to recover from packet loss through retransmission. This improves reliability but introduces additional delay and overhead.
UDP does not retransmit packets at the transport layer, making it faster and more suitable for applications where low latency is more important than guaranteed delivery.

Result

TCP retransmissions were observed in Wireshark after packet loss. UDP did not demonstrate TCP-style automatic retransmission, confirming the difference between reliable TCP communication and lightweight UDP communication.

4.DNS Traffic – Query Type, Response and Resolution Time

Aim

To capture DNS traffic using Wireshark, identify DNS query types such as A, AAAA, and MX, examine the response, and measure the resolution time.

Understanding of Concepts

DNS converts human-readable domain names into information used by network applications.
For example:
www.example.com
↓
DNS Server
↓
IP Address

Experimental Procedure

1. Open Wireshark.
2. Start packet capture.
3. Open Command Prompt.
4. Execute:

nslookup google.com

5. Stop the capture.
6. Apply the filter:

dns

7. Select the DNS query packet.

8. Expand the Domain Name System section.
9. Identify the query type and corresponding response.

DNS Query Types

Query Type	Purpose	Example Response
A	IPv4 address	142.250.x.x
AAAA	IPv6 address	2607:f8b0::
MX	Mail server	mail.example.com

Example Wireshark Observation

DNS Query:
Name: google.com
Type: A

DNS Response:
Answer: IPv4 address
For an IPv6 query:
Type: AAAA
Answer: IPv6 address
For mail server lookup:
Type: MX
Answer: Mail exchange server

Measuring Resolution Time

Wireshark provides timestamps for packets.
The DNS resolution time can be calculated approximately as:
DNS Resolution Time = Response Time − Query Time
For example:
Query time = 2.350000 s
Response time = 2.365000 s

Resolution time = 0.015 s
= 15 ms

Why DNS Uses UDP

DNS commonly uses UDP because:

- It has low overhead.
- No connection establishment is required.
- Queries are usually small.
- It provides fast request-response communication.
- It is efficient for large numbers of DNS queries.

DNS can also use TCP, particularly when responses are too large for the normal UDP exchange or for certain DNS operations.

Analysis of Results

The Wireshark capture shows that the DNS client sends a query and receives a corresponding response from the DNS server. The query type determines the information requested, such as IPv4, IPv6, or mail-server information.

The short query-response exchange demonstrates why UDP is generally efficient for DNS.

Result

DNS traffic was successfully captured in Wireshark. The **query type, DNS response, and approximate resolution time** were identified, and the reason for using UDP for typical DNS queries was analyzed.